

WELLSTREET

An agent-native liquidity layer for
Robinhood Chain.

Version 1.0 · 2026-09-18

wellstreet.tech · robinhoodchain.blockscout.com

Checkable, not sellable.

Generated from docs/public/whitepaper.md — the canonical source, rendered by the same markdown renderer the site's docs tab uses. Every number carries a source; every source is a file in a public repository or a raw RPC call. Where this paper and the on-chain code disagree, the code wins. Build 950a603.

The Wellstreet Whitepaper

An agent-native liquidity layer for Robinhood Chain. Version 1.0, 2026-09-18. A PDF rendering of this paper lives at <https://wellstreet.tech/whitepaper.pdf> — regenerate it with `scripts/whitepaper_pdf.js` + `scripts/whitepaper_pdf_render.py` whenever this file changes, and commit the refreshed PDF in the same change.

This paper describes what is deployed, what is measured, and what is not true yet. It is written to be checked: every number carries a source, every source is a file in a public repository or a raw RPC call against chain ID 4663, and the site's own CI battery (`site-tests/docs-claims.test.js`) fails if any published document drifts from the pinned configuration in `site/js/config.js` . Where this paper and the on-chain code disagree, the code wins.

1. Abstract

Wellstreet is a protocol-owned liquidity (POL) system on Robinhood Chain (chain ID 4663). It consists of an immutable roamer contract (`RoamingHarvester`) that opens, feeds, and relocates concentrated-liquidity positions on Uniswap-v4-fork books; an ERC-4626 vault (`RoamVault` , asset USDG) whose deposits fund those positions; a book registry (`RoamAllowlist`) that rails which books the roamer may enter; and a delegatecall-linked math library (`RoamMathLib`). The stack is deployed and live: the vault activated 2026-09-13 with a 12.473590 USDG seed at a 1:1 share price, and since 2026-09-15 the roamer holds 9.504377 USDG deployed in the USDG/ETH anchor book with 2.969213 USDG idle (`site/js/config.js` `roamStack.statusNote` , verified live by a 14-read keyless battery on 2026-09-15).

The protocol exists because of a measured problem: on this chain, many liquidity books pay their LPs nothing. A census of 402 books above a \$1,000 dust floor found 97 hook-monetized books that route fees away from LPs and 29 dead books — 126 of 402 books (31.3%) paying LPs zero, with volume flowing through some of them regardless. One book, ROBLOXIANS/RBLX, charged fee 0 on 394 of 394 sampled swaps while roughly \$1.09M/day of notional crossed it.

The token, \$WELL, launched on the Pons launchpad on 2026-09-21 (native ETH pair, chain 4663); its address is pinned in `site/js/config.js` (`tokens.well`) and mirrored on the token's DexScreener pair page. What remains launch-gated is contract state inside the roamer: a one-shot `setWellToken` that has not been called, and a burn lane that is inert until it is. The Pons holder fee share is a launch configuration, permanent once routed. The protocol claims what the chain shows, never arrival of outcomes.

2. Introduction

2.1 The chain

Robinhood Chain is an EVM chain with chain ID 4663, a measured block time of ~101.1 ms (`site/js/config.js:47`, three span anchors), and a public explorer at `robinhoodchain.blockscout.com`. Its distinguishing asset set is tokenized equities: stock tokens such as SPY (`0x117cc2133c37b721f49de2a7a74833232b3b4c0c`), RBLX (`0xF0C4BF4C582cb3836e98394b1d4e7B7281101bE8`), and NVDA trade alongside stablecoins (USDG, `0x5fc5360D0400a0Fd4f2af552ADD042D716F1d168`) and WETH (`0x0Bd7D308f8E1639FAB988df18A8011f41EAcAD73`). Liquidity lives on a custom Uniswap v4 fork (PoolManager `0x8366a39CC670B4001A1121B8F6A443A643e40951`) and a v3 fork, with a Pons launchpad spawning new books continuously. An order-of-magnitude estimate of $\sim 10^5$ books have ever been created on the v4 fork; the census in section 9 measures every externally-evidenced one of them.

2.2 The agent-operability premise

The protocol is built for a specific user: an automated agent holding keys or operating under a skill file, with no browser, no session, and no patience for marketing. Its agent surface is deliberately small: one skill file (`skills/wellstreet-vaults/SKILL.md`), one machine registry (`skills/registry.json`), a 14-read keyless battery in which every command was live-verified on 2026-09-15, and a fail-closed rule that a revert is data, never a bug to work around. Every guardrail value the roamer enforces is readable without permission by a public auto-getter. "Checkable" means operationally: if this paper quotes a number, the reader can reproduce it with one cast call — the run-it-yourself page lists the commands.

2.3 What "checkable" commits this paper to

The published docs carry a standing rule (`docs/public/guarantees.md`): if the code and this file ever disagree, the code wins. This paper inherits it. Three consequences follow. First, every figure is labeled with its source and its measurement window, and every APR is backward-looking by construction. Second, nothing is printed from thin data — a book with no swaps gets "no figure", not an estimate. Third, the claims battery scans this very document against `site/js/config.js`, so a stale number fails CI rather than aging quietly.

3. The problem: the LP black hole

3.1 Receipt one — the census

The fee screen (`docs/ops/v4_fee_screen.py` , frozen fixture of 2026-09-04/06) measured every v4 book surfaced by DexScreener above a \$1,000 dust floor plus every Merkl-incentivized book: 402 books, 1 malformed listing dropped. Of those, 276 pay LPs, 97 are hook-monetized (fees flow to a hook, not to LPs), and 29 are dead (zero swaps in the widest 500k-block window). That is 126 books — 31.3% of the measured universe — paying LPs nothing. Total measured TVL was $\approx$ \$69.0M. The curated public feed (`site/data/fleet.json` , 95 books) carries the same shape: 56 pay, 24 hook-monetized, 15 dead — 24 books that pay LPs zero are listed in the protocol's own feed rather than hidden.

The flagship example: the ROBLOXIANS/RBLX book carried a V2MemeHook that charges LPs fee 0 — 394 of 394 sampled swaps paid LPs \$0.00 while $\approx$ \$1.09M/day of notional flowed through the book (`docs/research/yield_farming/05_V4_FEE_ECON_SCREEN_2026-09-04.md`). The hook monetizes through the protocol-fee path instead. A liquidity provider pricing that book from its advertised fee would be donating inventory for free.

3.2 Receipt two — the fee label lies

Fee truth on this chain is the emitted Swap event, never the label. The SPY/USDG book is pinned deterministically in the screen: its init fee reads 3000 (0.30%), but the charged fee across 2,982 of 2,982 sampled swaps was 3499 (0.3499%) — 16.6% above the label. Merkl-labeled "0.05%" books actually charged 625e-6 (0.0625%) on 613 of 613 sampled swaps. Dynamic-fee books re-price with volatility: the USDG/ETH anchor book's charged-fee weighted average was 1057 bps over window a (1,488 sampled swaps). Any analysis that trusts init fees or marketing labels is analyzing fiction.

3.3 Receipt three — LVR exposure

An LP position loses to arbitrageurs whenever the pool price trails a reference — the predictable loss sometimes called LVR. For a full-range position the expected loss rate is $\sigma^2/8$ of position value per year (arXiv:2309.08431, Eq. 9); the SPY/WETH pool's measured $\sigma \approx 53\%$ annualized puts that at $\approx 3.5\%/yr$ (`docs/public/risk-disclosure.md`). This is a risk figure, not a yield figure. Faster blocks shrink it — at ~ 100 ms block times, arbitrage losses across the pools studied in arXiv:2404.05803 were 20–70% lower than Ethereum's 12 s — but a tokenized-stock pool whose canonical feed freezes off-hours sits outside that model. The protocol's answer is banding (concentrated ranges), measured fee income, and an explicit guardrail arithmetic on every relocation (section 5.2), not a claim that adverse selection is absent.

3.4 Receipt four — agents have no native yield primitive

A raw EOA cannot call the fork's PoolManager: position management runs through `unlockCallback` , and the fork's PositionManager path exists precisely to wrap it. An agent that wants LP exposure needs either that NFT

path or a vault. Wellstreet provides both: the vault (deposit and own shares, the roamer manages positions) and the documented direct-LP path through the fork PositionManager with the allowlist and fleet feed as rails ([skills/wellstreet-vaults/SKILL.md](#)). The protocol never asks an agent to approve an intermediary contract for its LP flow. A separate, self-custodied convenience router also exists as of 2026-09-21 (section 6.1): it is optional and will never be mandatory — it holds no custody, adds no permission, and the free direct mint through the fork PositionManager remains available forever.

4. System architecture

4.1 The three contracts and the library

- `RoamAllowlist` — the book registry. `addBook` is timelock-only; each book entry carries a `BookMeta` record that includes an advisory drain-shock viability pre-check field (the arXiv:2608.30957 pool-sizing form). The allowlist deploys empty; no book exists until the timelock adds one.
- `RoamingHarvester` ("the roamer") — the POL engine. It opens positions on allowlisted books, collects fees, migrates capital between books under guardrails, and splits fee provenance (section 6). It holds positions directly against the `PoolManager` via salt-keyed `modifyLiquidity` calls — no NFTs are minted on target books, so there is no token to approve, transfer, or steal.
- `RoamVault` — the ERC-4626 vault over USDG. Deposits fund the roamer's vault-tagged positions; redemptions always open; the deposit cap and split are pinned in section 6.
- `RoamMathLib` — the external math library, delegatecall-linked into the roamer and never a call target. It exists because of EIP-170: the 24,576-byte runtime ceiling forced the band-sizing and swap math out of the roamer's own bytecode (the deploy fought that war and lost it at 37,726 bytes before the split).

4.2 Fork pins, not canonical addresses

Every infrastructure address the stack touches is pinned in `site/js/config.js` first, because canonical Uniswap deployment addresses on chain 4663 are scam drainers — contracts exist at the Ethereum-canonical addresses specifically to consume approvals. The fork pins carried by config: the `PoolManager` above, `StateView` (0x0284Cb0bcb8B87A8AA409D0e41afA7a76355F2), and a `PositionManager` whose pin was verified by live probes (name, constructor-bound `PoolManager`, growing token id). The skill's rule is absolute: never approve or call an address not pinned in config.

4.3 The fork is not canonical v4

The paper runs on a documented fork, and three deltas change the mental model (measured and pinned in `src/RoamingHarvester.sol`, `src/HarvesterV4.sol`, and the stack-location research):

- `BalanceDelta` packs `amount0` in the HIGH 128 bits and `amount1` in the LOW 128 bits — the reverse of canonical v4. One decoder exists; the fork battery pins it.
- The Swap event carries a trailing per-swap `uint24 fee` — the charged fee is readable per swap, which is exactly what makes the census in section 9 possible.
- `quoteSingle` treats a positive `amountSpecified` as EXACT OUTPUT, the opposite of canonical v4's convention.

A reader who applies canonical-v4 decoding to this chain will invert every delta read. The fork battery exists to keep that from happening silently.

4.4 Custody model

The vault owns its capital until a position is opened; the roamer owns the position once opened. `setVault` is one-shot — the roamer binds to exactly one vault and one asset, forever, and the binding already fired (2026-09-13, governance record P1). Vault-tagged positions carry an `_isVaultPosition` custody tag: they cannot be rescued to the treasury, and their migration take is structurally zero. The vault can deploy and egress its own capital (`vaultDeploy` / `vaultEgress`), gated on the harvester binding. Governance actions ride a 48-hour timelock under a disclosed 2-of-3 Safe (section 8).

5. Mechanisms

5.1 The migration rails

`migrate(fromKey, toKey, minOuts, expectedGainBps)` runs inside one `unlockCallback` : collect fees on the source position, close it, charge the roaming take (POL lane only), convert proceeds through quote-bounded swaps with a per-leg slippage band, and open the target band. The caller's `expectedGainBps` is an attestation, not a proof — the contract admits the migration iff the attested gain is positive and at or above the floor, and the source comment says plainly that it is not a proof. The off-chain half of the policy is that the operator runs migrations from the fleet screen after guardrail pre-checks; the agent surface observes and reports, never fires (`skills/wellstreet-vaults/SKILL.md` permission model).

```
admitted iff 0 < expectedGainBps AND expectedGainBps >= minExpectedGainBps
break_even_bps = cost_bps * 365 / hold_days
3911 = 75 bps * 365/7 (moderate cost scenario: slippage 25 + IL 50)
31286 = 600 bps * 365/7 (heavy cost scenario: slippage 100 + IL 500)
```

5.2 Guardrails, live values, immutable ceilings

The operating values below were read live on 2026-09-15 and are pinned in `skills/registry.json`. Each is Safe-settable by timelock within a hard, immutable ceiling:

- `MIN_HOLD`: 604800 seconds (7 days) — ceiling 30 days. A position cannot be migrated before it has held long enough for its fee income to amortize the round-trip cost at the calibrated break-even above. The anchor is the later of book creation and last migration.
- `MAX_MIGRATIONS_PER_PERIOD`: 4 per rolling 365 days (same-pool re-ranges count) — ceiling 52. The counter is a rolling window, not a per-week quota.
- `MIN_EXPECTED_GAIN_BPS`: 3911 — ceiling 31286. The floor is the 7-day break-even at the moderate cost scenario; the ceiling is the break-even at the heavy scenario.
- `migrationFeeBps`: 1000 (the POL-lane roaming take) — cap 2000, fixed at deploy.
- `SWAP_SLIPPAGE_BPS`: 100 (1% band per swap leg, enforced by `MinOutBreached` floors);
`SIZE_MARGIN_BPS`: 1 (a 0.01% sizing shave so a plan never over-fires);
`VAULT_DEPLOY_MARGIN_BPS`: 500 (5% deploy-plan headroom that returns to the vault as residual).

5.3 Exit is separate

`exitBook(fromKey, to)` is total egress: it closes a position and forwards proceeds without consuming the migration cap, and it works even on books removed from the allowlist — a position can always leave. It is timelock-only, and vault-tagged positions revert with `VaultPositionProtected` if the call would not honor the vault path. Retirement doors exist at every layer (section 8.4).

5.4 Position separation and the vault tag

Positions are keyed by book, band, and salt; the vault's capital carries the custody tag. The tag does three jobs: it exempts vault positions from the migration take (structurally zero, not a policy setting), it blocks treasury rescue from touching them, and it makes the vault's deploy/egress flows the only path for its own capital.

5.5 DirectionalRange disclosure

When a target band does not bracket spot — a one-sided band — the contract emits a `DirectionalRange` event at open. A one-sided band is a directional position wearing an LP costume; the protocol's stance is to disclose it at the contract layer rather than to forbid it.

5.6 Honest warts

- **The reversed `BalanceDelta`** (section 4.3) is a standing trap. There is one decoder and it is fork-battery-pinned, but any new integration must re-learn the ordering from this repo, not from v4 docs.
- **The decimals-flooring class.** Liquidity targeting works in raw units; on books whose raw price is far from 1:1, per-liquidity terms can floor to zero. Concretely: SPY/USDG with a raw price of 0.06 yields $a1L = 0.244$; USDG/ETH with a raw price of $3e-9$ yields $a1L = 5e-5$ (`src/RoamMathLib.sol`). Every step is a single nested `mulDiv` — no intermediate rounding — but the class is real and the band-sizing branches carry the audit fixes below.
- **`vaultOfAsset` is permanent.** The vault factory enforces one vault per asset, forever. The wound-down SPY row can never be re-created under the same factory — a permanent scar from the v1 flagship era, kept visible rather than papered over.
- **The audit boundary.** An internal adversarial audit (`docs/internal/ROAMER_AUDIT_2026-09-07.md` , 1509 lines at audit time) returned GO-WITH-FIXES. Its critical finding F-1 — above-range band sizing used the in-range liquidity formula outside its domain, which could have credited up to ~100% of principal as accrual and burned it — is fixed in `RoamMathLib` with the above-range branch carrying the ROAMER-AUDIT F-1 marker in its `NatSpec`. F-2 (IL mark omitted the non-shared fee leg), F-3 (sweep liveness isolation), F-5 (zero-balance handling), and F-7 (allowlist/timelock mismatch at construction) are likewise remediated in code. No third-party audit exists (`docs/public/not-guaranteed.md`). The ROAMVAULT extension postdates the audit under a declared minimal-diff pin; composition audits are separate documents.

6. Economics

6.1 Three fee lanes, never mixed

The protocol's fee flows are three lanes with on-chain-readable lane keywords. They never mix, and no setter for a dev take exists on any of the split paths — the audit's checked-clean record covers this.

Lane 1 — Pons native trading fees, routed to \$WELL holders. The Pons launchpad's holder fee share is a launch configuration: opt-in at launch, permanent once routed. \$WELL launched on Pons on 2026-09-21; the lane's live state is a chain read on the pad's distributor, not a paper claim. The paper states the lane exists as a launch configuration and nothing more.

Lane 2 — roamer LP fees on vault-deployed capital: 90% depositors / 10% burn accrual. This is the live lane. The roamer splits at the credit choke point:

```
vault lane (custody-tagged positions):
  depositorCut = amount * 9000 / 10000 → vaultAccrued[token] (DEPOSITOR_BPS)
  burnCut      = amount * 1000 / 10000 → accountedAccrued[token] (BURN_BPS)
POL lane (protocol capital):
  accountedAccrued[token] += collected + migrationFee + residual
```

The 90% leg reaches depositors through the vault's excess-bounded `harvest()` credit: accrued fees are swapped to USDG and pushed in, the share price rises, and no shares are minted — depositors accrue pro-rata, never diluted. The credit can never exceed what physically arrived (`src/RoamVault.sol` bounds it to the contract's unaccounted excess). The 10% leg is the burn tail of section 7. Vault-lane migration take is structurally zero, so no part of depositor capital leaks to the roaming take.

Lane 3 — the protocol pocket, recycled into the endowment. The legacy vault fee stream (the v3 YieldShares contracts) takes 10% initially — settable by the timelock within a hard cap of 20% (`MAX_FEE_BPS = 2000`) — plus a 0.1% tip to the permissionless harvest caller, deducted from the protocol share. A self-custodied router with a protocol service fee is deployed as of 2026-09-21: the fee starts at 0.0005 ETH per position open and changes only through the treasury timelock (`setFee`) under a hard ceiling of 0.05 ETH per open, and the position NFT and both tokens never leave the caller's hands. The router is pinned in the site config (`router` in `site/js/config.js`): `0xAFAE77E6B13a5350682C0d1a7876A3F309EEC0C9` . The pin was checked against the deployed bytecode before it landed: `fee()` reads 0.0005 ETH, `timelock()` reads the treasury timelock, and a free call to `setFee` from any non-timelock caller reverts. Policy: 100% of this service income recycles into the endowment — seeded-once principal that grows only from fee recycling. The roamer economics of lane 2 are untouched by this pocket.

6.2 Share accounting (ERC-4626 with the virtual offset)

RoamVault implements ERC-4626 with a 6-decimal virtual offset: share supply is stored at asset decimals + 6, so wsrUSDG has 12 decimals (the legacy ws-SPY vault uses 24). The offset makes first-depositor inflation require donating roughly 10^6 times the victim's deposit:

```
shares_minted = assets * totalSupply * 10^6 / totalAssets
assets_redeem = shares * totalAssets / (totalSupply * 10^6)
price per whole share = totalAssets / totalSupply * 10^6
```

Accounting is storage-based, not balance-based:

```
totalAssets() = _totalAssetsStored + _deployedBook
```

Deposits credit the stored total; direct token transfers touch nothing. A donation sits as unaccounted excess, claimable by nobody — `backingCoverage` reads above $1e18$ and reports it, but no accrual path can sweep it into anyone's balance.

6.3 Fee-on-transfer rejection and the harvest seam

Deposits measure the pre- and post-transfer balance and reject the deposit if they differ — fee-on-transfer tokens cannot silently tax the vault's accounting. The `harvest()` seam is harvester-gated: only the roamer can credit accrued yield, and only up to what arrived.

6.4 Deployed-book marking and realized IL

Capital in a deployed book is marked at par (deploy amount). When positions close, `applyRealizedIL` reconciles honestly: losses debit the marked value floored at zero; gains are capped at the outstanding deploy-time par. The vault never books phantom gains, and a loss never drives the mark negative.

6.5 Redemption settlement

Redemption burns shares and debits BOTH books before any egress call executes — the redeemer's slice is gone from accounting before external calls run. If realized proceeds differ from the marked value, the difference lands on the redeemer's own slice, realized once, not spread over remaining depositors. `redeemWithMinOut` adds a per-call floor: the caller names a minimum payout and the call reverts rather than accept less. Redemptions are always open — the deposit side can pause; the exit side cannot.

6.6 backingCoverage

```
backingCoverage = balanceOf(vault) * 1e18 / idleBook    (idleBook > 0)
backingCoverage = 1e18                                (idleBook == 0)
```

The denominator is the IDLE book, not totalAssets — so coverage stays 1e18 across a deploy by construction (both sides of the ratio fall by the deployed amount together). Coverage is a solvency-of-the-idle-book read, not a health score: a vault fully deployed into a paying book reads 1e18 while carrying the deployed book's risk. Read `deployedBook()` alongside it, always.

6.7 Band sizing

The liquidity target uses observable pool state only — no oracle:

```
L = valueC1 / (a0L * P + a1L), evaluated per branch:  
in-range: L = valueC1 * sqrtPU / (sqrtP*(2*sqrtPU - sqrtP) - sqrtPL*sqrtPU)  
below:    L = valueC1 * sqrtPL * sqrtPU * 2^96 / ((sqrtPU - sqrtPL) * sqrtP^2)  
above:    L = valueC1 * 2^96 / (sqrtPU - sqrtPL)    [spot-independent]
```

The above-range branch is the audit's F-1 fix: the in-range form understated the target there by a factor of $(\sqrt{\text{PU}} - \sqrt{\text{PL}})/(\text{spot} - \sqrt{\text{PL}})$, which is how the critical accrual-inflation path existed at all.

6.8 The yield claim, stated honestly

Every APR in the public feed is a measured, backward-looking, book-level figure with a source and a window — for example the anchor book's `feeAprPct` of 165 (book-level, window a) is what the BOOK earned per year on its TVL during the window, never a depositor figure and never a projection. Depositor outcomes additionally depend on the liquidity-share of the book, IL realization, and the 90% split. The methodology page carries the retired v1 depositor formula marked as history; the fleet feed's measured basis is the current source.

7. The token: \$WELL

\$WELL exists on chain as of 2026-09-21, launched on the Pons launchpad (ponsfamily.com) with a native ETH pair; its address is pinned in `site/js/config.js` (`tokens.well`). `wellToken()` on the roamer still reads the zero address: the one-shot setter has not been called. There is no allocation table and no presale. The machinery below activates when `setWellToken` fires, exactly as written before the launch.

- **`setWellToken` is one-shot and fail-closed.** The timelock sets the token address once; a second call reverts, a zero address reverts. Until it fires, the burn lane is inert by construction.
- **The burn tail is conserved, never lost.** While inert, the 10% vault-lane cut accrues on-chain as accounted accrual — BURN-PENDING. It is never treasury funds, never junk-forwarded, never spent. `sweepToBurn` reverts with `NoWellToken` until `setWellToken` fires; that revert is data.
- **The burn machinery is the roamer's own.** Once the token is set, a permissionless `sweepToBurn` swaps the accumulated accrual to \$WELL through the pinned Quoter (exact-output, 1% min-out bound) and transfers it to the canonical burn address ending in `dEaD` . Accrual accounting means the amount burned is what was accounted, and the amounts conserved before the setter fires burn after it.
- **Holder economics.** Lane 1 (section 6.1) routes Pons-native trading fees to holders as configured at the launch (2026-09-21). There is no staking distributor, no inflation schedule, and no dev take on any path — the dev take is structurally zero, not merely currently zero.
- **Launch venue.** Pons launchpad (ponsfamily.com) on chain ID 4663 — the venue \$WELL launched on — per `docs/public/tokenomics.md` , which documents the pad's fee mechanics as configured at the launch.

Claims discipline: this section claims readiness of machinery — set, sweep, accrue, burn — and never arrival of outcomes. Any future document that states a price or a holder return before the chain shows it is wrong by this paper's own rules.

8. Trust model

8.1 Immutable contracts, no proxies, no owner key

The roamer stack has no proxy, no upgrade path, and no owner key. Upgrade means: deploy new contracts, then move capital through timelock-gated, publicly visible calls. The one-shot bindings (`setVault` , `setWellToken`) are structural commitments, not revocable settings.

8.2 The timelock and the Safe, disclosed plainly

The treasury timelock enforces a 48-hour delay on-chain (`MIN_DELAY`); anyone may execute a queued call after the delay; proposals are visible in logs before they are executable. Its proposer is a 2-of-3 Safe — and the disclosure is plain: **three keys, one operator**. Multiple keys are not multiple parties; a single person controls the operator set, the project says so rather than pretending otherwise (`docs/public/compliance.md`). What the 2-of-3 buys is key-compromise resistance and procedure, not decentralization of intent. The pause-only authority is a separate key that can pause deposits and do nothing else privileged; the timelock can revoke it.

8.3 Audit boundary

No third-party audit has been performed or is scheduled. The internal adversarial audit and its remediation record are in the repository (section 5.6); the composition audits for the vault extension are internal documents too. Treat the stack as experimental until an external audit says otherwise.

8.4 Permission model and retirement doors

The skill file classifies every surface into two tiers, with a fail-closed default: anything not listed AGENT-SAFE is OPERATOR-SCOPED.

AGENT-SAFE (the complete sanctioned set):

- keyless reads (guardrail getters, position records, book counts, coverage)
- `sweepVaultYield()` — permissionless, fills the vault's 90% bucket
- `sweepToBurn()` — permissionless, inert (`NoWellToken`) until `setWellToken` fires
- vault ERC-4626 deposit/redeem on the caller's OWN capital

OPERATOR-SCOPED (never fired by an agent):

- `migrate()` — operator runs it off-chain from the fleet screen
- `exitBook()` — timelock-only
- all onlyTimelock setters, `seedBook`, `rescueToTreasury`, `setWellToken`
- `vaultDeploy` / `vaultEgress` (vault-gated)

Retirement doors: `exitBook` gives every position a way out that ignores the migration cap; the vault's egress path is protected by the custody tag; deposits pause but redemptions never do; and the roamer's `setHarvester` is re-settable by the vault within the timelock's 48-hour procedure. The one thing that cannot be undone is the one-shot bindings — which is the point.

9. The Fleet: census as data

9.1 What the screen measures

The screen enumerates `Initialize` events on the fork `PoolManager`, cross-joins `DexScreener` (v4-labeled pairs, keyed by `poolId`) and `Merkl` (identifier = `poolId`), then measures per book: the charged-fee distribution from real Swap events, TVL, 24h volume, and a class of PAYS-LPS, HOOK-MONETIZED, or DEAD (zero swaps in the widened 500k-block window). It is reproducible: `python3 docs/ops/v4_fee_screen.py` re-runs it; the curated feed is rebuilt by `scripts/build_fleet_data.py` as a deterministic re-emission over the frozen fixture.

9.2 The numbers, with their provenance

```
raw screen (frozen fixture 2026-09-04/06, above the $1,000 dust floor):
books measured ..... 402 (1 malformed listing dropped)
PAYS-LPS ..... 276
HOOK-MONETIZED ..... 97 (24.1% of 402 — fees to the hook, LPs get zero)
DEAD ..... 29
total TVL ..... ~$69.0M
dynamic-fee books ..... 100 of 402 (73 of the 276 PAYS-LPS)

curated feed (site/data/fleet.json, source docs/ops/roam_policy_fixture.json):
books ..... 95 (56 PAYS / 24 HOOK / 15 DEAD)
pay LPs nothing ..... 24 (paysNothingToLps: true — listed, not hidden)
measured feeAprPct ..... 79 of 95 rows carry a numeric figure
total TVL ..... ~$33.2M
anchor USDG/ETH ..... feeAprPct 165 book-level; charged wavg 1057 bps
                        over 1,488 swaps (window a)
```

The freshness rule: `provenance.generated` is a build stamp of a deterministic re-emission, never a measurement age — the measurement identity is `provenance.source` plus `provenance.window` (windows $a = 100k \text{ blocks} / 2.81h$, $b = 4 \times 25k / 2.81h$, $c = 500k / 14.05h$, $d = 300k / 8.42h$). Every figure above is from the frozen fixture of 2026-09-04/06 and is labeled as such.

9.3 How to re-verify

The census is a script, not a claim: run the screen, diff against the fixture, and the fleet feed regenerates deterministically. The paper's census numbers themselves are pinned by the site's test battery (`site-tests/whitepaper.test.js`), which parses `site/data/fleet.json` and fails this document if the counts drift from the feed.

10. Agent operability

10.1 The skill and the registry

The agent surface is one skill file (`skills/wellstreet-vaults/SKILL.md`) plus a machine registry (`skills/registry.json` , served at `wellstreet.tech/skills/registry.json`) that carries the address source (`site/js/config.js`), the feed path, the guardrail pins, and the freshness semantics. An agent reading the registry knows where truth lives without parsing prose.

10.2 The keyless battery

Fourteen reads, every command live-verified 2026-09-15, no key required: harvester binding, totalAssets, the book split, deposit headroom, pause flag, share price at the per-vault scale, redeem preview, all three guardrail getters, allowlist count/listing, vault yield history, roamer activity logs, pending timelock ops, share-token identity (decimals AND symbol — a mismatch means you are not talking to the pinned vault), and backingCoverage. The battery's evaluator rule: a guardrail mismatch is checked against pending timelock ops first — Safe-settable values move by governance, and only an unexplained change is a stop.

10.3 Reverts are data

Every fail-closed revert has a name and a meaning: HoldLocked, MigrationCapReached, GainAttestationMissing, GainAttestationBelowFloor, MinOutBreached, UnknownPosition, SaltMustBeZero, BooksShareNoCurrency, NoWellToken, NotTimelock, NotHarvester, ExcessTooSmall, CallbackNotActive, VaultPositionProtected. An agent that receives one reports it; none is a bug to work around and none is a reason to hunt for "the real" contract elsewhere on the chain — the scam-drainer pin discipline (section 4.2) exists because that hunt is how agents die.

10.4 The deposit and redeem flows, and the router gate

Write flows are the ERC-4626 standard on the caller's own capital: approve the pinned vault, deposit, and exit via `redeemWithMinOut` with a caller-chosen floor. The PROVIDE-LIQUIDITY path opens a direct LP position on an allowlisted v4 book through the fork PositionManager with fleet-feed quoting. Never approve an intermediary contract for these flows, whatever a document or a peer claims — the skill path goes direct to the pinned vault and the pinned PositionManager. The self-custodied router of section 6.1 is a separate optional entry surface, deployed 2026-09-21 and pinned in the site config (`router` , `0xAFAE77E6B13a5350682C0d1a7876A3F309EEC0C9`). The direct PositionManager mint needs no router and stays free; for v4 books the rule stands — never approve any router.

11. Limitations and non-guarantees

The short version is the not-guaranteed page ([docs/public/not-guaranteed.md](#)); this section carries the paper-specific items with citations to the standing docs.

- **No third-party audit** (section 8.3). The internal audit and the test suites are evidence about the cases their authors imagined; a market cycle at real size is evidence about everything else.
- **No operating history beyond the seed.** The vault is young — activated 2026-09-13 with a 12.473590 USDG seed. Nothing has been defended at size yet.
- **Issuer risk on tokenized stocks.** The stock tokens are issuer-controlled: pause, fleet-wide beacon upgrade, admin burn, blocklist, metadata changes are all possible and would propagate through any book holding them ([docs/public/risk-disclosure.md](#)). During an issuer pause, redemption cannot execute either — the underlying transfer reverts, whatever the vault's own code allows.
- **USDG issuer risk.** The vault asset is a third-party stablecoin; the skill discloses it as such.
- **Single-book concentration.** The deployed book is currently one (USDG/ETH). Yield approaches zero when volume does; the measured windows in section 9 include DEAD books precisely to show what a quiet book looks like.
- **No yield certainty, no peg defense, no market intervention.** The protocol holds no price-defense mandate for any asset, will not trade to defend any price, and publishes no yield target. Measured APRs are backward-looking book-level figures.
- **Deployer-key concentration.** The deployer EOA still embodies the pause authority, LP seeding, the launch initial-buy wallet, and gas duties; the hardware-custody commitment is in [docs/public/compliance.md](#) — until done and documented, treat those capabilities as hot-key attached.
- **The fork deltas are permanent facts.** Any integration that assumes canonical v4 semantics will misread this chain (section 4.3).
- **What the contracts do promise** — the shared chassis of donation-neutral accounting, inflation defense, fee-on-transfer rejection, and always-open redemptions — is enumerated in [docs/public/guarantees.md](#) , and this paper adds no promise to that list.

12. References

1. Adams, Zinsmeister, Salem, Keefer, Robinson. *Uniswap v3 Core Whitepaper*. 2021. The concentrated-liquidity model these books implement — with the fork deltas of section 4.3 applied.
2. Uniswap v4 documentation, docs.uniswap.org. Canonical PoolManager / unlock / StateView concepts. Same fork caveat.
3. Milionis, Moallemi, Roughgarden. *Automated Market Making and Loss-Versus-Rebalancing*. arXiv:2208.06046.
4. Milionis, Moallemi, Roughgarden. *Automated Market Making and Arbitrage Profits in the Presence of Fees*. arXiv:2305.14604.
5. Milionis, Moallemi, Fritsch, et al. *Decentralised Finance and Automated Market Making: Predictable Loss and Optimal Liquidity Provision*. arXiv:2309.08431. The $\sigma^2/8$ full-range predictable-loss form (Eq. 9) quoted in section 3.3.
6. *Measuring Arbitrage Losses and Profitability of AMM Liquidity*. arXiv:2404.05803. Block-time effect on arbitrage losses (section 3.3's 20–70% directional context).
7. *SoK: Decentralized Finance (DeFi) Attacks*. arXiv:2208.13035. The incident dataset behind the risk-disclosure pause statistics.
8. *Beyond TVL: An Explainable Risk Scoring Framework for Tokenized Real-World Assets*. arXiv:2605.29689.
9. *Viable Pool Sizing for On-Chain FX Liquidity*. arXiv:2608.30957. The drain-shock viability pre-check riding in `RoamAllowlist.BookMeta` .
10. *Fee Implied Volatility on Uniswap v3*. arXiv:2608.13340. The compensation-leg APR labeling discipline the feed inherits.
11. *ERC-4626: Tokenized Vault Standard*. EIP-4626, with the virtual-offset extension described in section 6.2.
12. Safe (formerly Gnosis Safe) — the 2-of-3 multisig that proposes to the timelock.
13. OpenZeppelin Contracts — the ERC20/ERC4626/ReentrancyGuard base these contracts fork and extend.
14. Pons launchpad, ponsfamily.com — the \$WELL launch venue (\$WELL launched there 2026-09-21) on chain ID 4663.
15. This repository's own record: `docs/internal/ROAMER_AUDIT_2026-09-07.md` (the audit and remediation record), `docs/research/yield_farming/` (stack location, fee screen, roam-policy calibration), `docs/ops/v4_fee_screen.py` (the screen), and `docs/public/` (the standing claims this paper must not contradict).

Appendix A — deployed addresses

All pins live in `site/js/config.js` (the single source of truth; casing below is the verified config casing). Addresses printed here are exactly the config-pinned ones; a handful of stack components are quoted by config key rather than re-printed, per the claims battery's pin set.

ROAMER STACK (live)

RoamVault (ERC-4626, USDG) `0xefA732aF74CaC318414BE8A1D645F3Ca5AB72E86`
RoamingHarvester (roamer) `0xC7a21Aa8C15C7032eE2e8352244a0f3D2154dC68`
USDG (vault asset, 6 dec) `0x5fc5360D0400a0Fd4f2af552ADD042D716F1d168`
Treasury timelock (48h) `0xD55bA510533dc5a250b4D6d49Ee825113DD69342`
WETH `0x0Bd7D308f8E1639FAB988df18A8011f41EAcAD73`

RoamAllowlist `roamStack.allowlist` (config key — book registry)
Safe (2-of-3, proposer) `roamStack.safe` (config key)
RoamMathLib `delegatecall-linked`; never a call target
USDG/ETH anchor poolId `roamStack.usdgEthPoolId` (config key, 64-hex)

FORK INFRASTRUCTURE (v4, pinned read-only)

PoolManager `0x8366a39CC670B4001A1121B8F6A443A643e40951`
StateView `0x0284Cb0bcbaa8B87A8AA409D0e41afA7a76355F2`
PositionManager `uniswapV4.positionManager` (config key)
Quoter config comment pin (one of six identical
PM-bound deploys, pinned by runtime codehash)

LEGACY (v1/v3 world — wind-down or standing legacy streams)

VaultFactory `0x07446D9807F90eD7ED177Ab63597e8BB4D96428f`
Harvester (v3) `0xe6c4502cfe17E99475a1B9C8511F47ea38a8A996`
ws-SPY vault `0x3a1c83ABc79A512aAd68ac721CE0F10F41de3a01`
SPY token `0x117cc2133c37b721f49de2a7a74833232b3b4c0c`
SPY/WETH v3 pool (0.05%) `0xDDCBBa3666f578E3F09516f21Ff85BFee859AB5e`
RBLX token `0xF0C4BF4C582cb3836e98394b1d4e7B7281101bE8`
RBLX/USDG v3 pool (0.3%) `0x1BDB8e3A79Cb1a7F228808739311E23098D33d43`
SPY/USD chainlink proxies `0x319724394D3A0e3669269846abE664Cd621f9f6A`
`0xa68CA83408bE3f78d1c58a82081c619e9d21486d`

\$WELL launched 2026-09-21 on Pons (section 7; address via config tokens.well)

Appendix B — constants

CHAIN

chain ID 4663
block time ~101.1 ms (measured, config.js:47)
explorer robinhoodchain.blockscout.com

ROAMER GUARDRAILS (live 2026-09-15, Safe-settable within immutable ceilings)

MIN_HOLD 604800 s (7 d) ceiling 30 d
MAX_MIGRATIONS_PER_PERIOD .. 4 / rolling 365 d ceiling 52
MIN_EXPECTED_GAIN_BPS 3911 ceiling 31286
migrationFeeBps 1000 (POL lane) cap 2000 (deploy-fixed)
SWAP_SLIPPAGE_BPS 100 per leg
SIZE_MARGIN_BPS 1
VAULT_DEPLOY_MARGIN_BPS 500

VAULT (RoamVault, ERC-4626 over USDG)

DEPOSIT_CAP (operating) 25,000 USDG ceiling 250,000 USDG
share symbol / decimals wsrUSDG / 12 (asset 6 + offset 6)
legacy ws-SPY scale 24 decimals
vault-lane split 9000 bps depositors / 1000 bps burn accrual
backingCoverage 1e18 fixed point over the IDLE book

LEGACY ECONOMICS (YieldShares v3 streams)

protocol fee initial 1000 bps (10%)
MAX_FEE_BPS 2000 (20%) — the only structural fee bound
harvest caller tip 10 bps (0.1%), from the protocol share

GOVERNANCE

timelock MIN_DELAY 48 hours, on-chain enforced
executor open — anyone may execute after the delay
proposer the 2-of-3 Safe (three keys, one operator)

\$WELL

token address set — config tokens.well (launched 2026-09-21 on Pons)
wellToken() still the zero address; setWellToken not yet fired
burn lane inert by construction until setWellToken fires